CSC 113 Phase 1 Project Report

By:

- **Alfarouq Alshukri – 446109472**

- **Abdulmohsen Alahmari – 446102155**

Instructor: Dr. Khaja Mustafa

Github repo: https://github.com/mhsn555/CSC-113-Project-

Introduction

This project represents a School Management System, developed as the core term project for the CSC 113 course. We chose to model a school environment because it provides a highly natural, real-world hierarchy to demonstrate object-oriented programming concepts. It allows us to cleanly map out roles (like students and staff) and organizational structures (like classrooms and clubs) into interactive code.

The console-based application manages the relationships and data of different objects within a school. By applying aggregation, composition, inheritance, abstraction, polymorphism, and interfaces, the system provides a robust architecture where different users can interact with the database according to their designated roles.

Core Services Provided

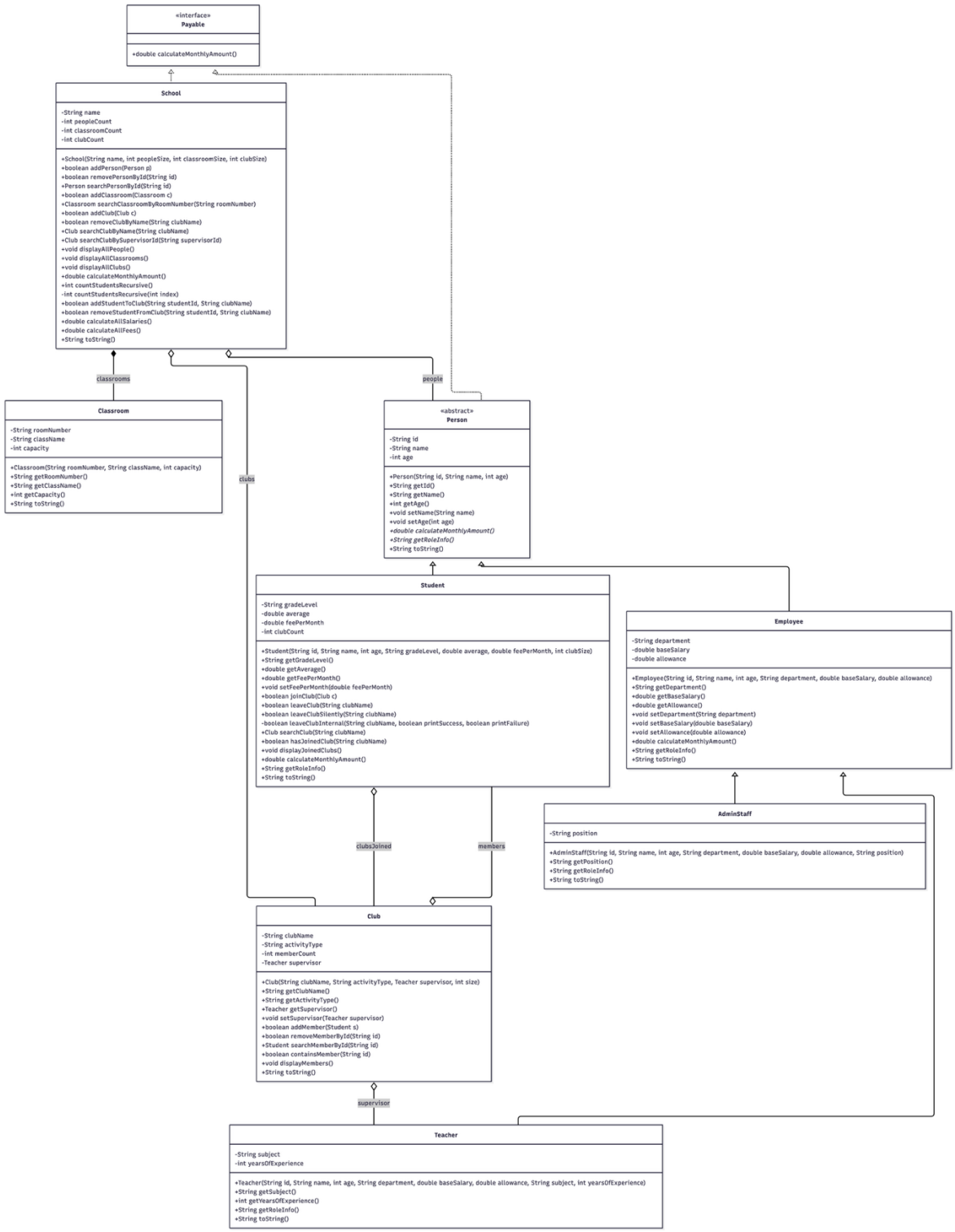
- **Object Management:** Management of school personnel, classrooms, and student clubs.
- **Relationship Synchronization:** Handles relationships, such as assigning a student to a club while simultaneously updating both the club's roster and the student's personal record.
- **Financial Processing:** Automatically calculates school-wide financial totals, including combined student fees and total employee salaries.
- **Role-Based Access:** Provides a customized console menu system that restricts and tailors functionalities based on whether the logged-in user is a Student, Teacher, Employee, or Admin.

General Architecture

The system revolves around the `School` class, which acts as the primary data container and coordinator. The system categorizes all individuals under an abstract `Person` class, which branches into `Student` and `Employee`. The `Employee` class further branches into `Teacher` and `AdminStaff` to handle specialized administrative and supervisory roles. `SchoolTest` acts as the frontend driver, providing the interactive menu loops.

Full UML Diagram

The following UML diagram presents the overall class structure, inheritance hierarchies, and relationships across the system:



Implementation And Design Summary

This section highlights the architectural purpose of each class and its most critical operational logic.

1. Interfaces & Abstraction

- **`Payable` Interface:** Defines a strict contract containing the `calculateMonthlyAmount()` method. It is implemented by classes that deal with money (`Person` and `School`) to ensure polymorphic financial calculations across completely unrelated objects.
- **`Person` Class:** The abstract base class that encapsulates fundamental identity data (ID, name, age). It forces all subclasses to implement their own version of `calculateMonthlyAmount()` and `getRoleInfo()`, ensuring polymorphic behavior throughout the personnel arrays.

2. The Inheritance Hierarchy

- **`Student` Class:** Extends `Person`. Beyond standard academic attributes, it manages an array of `Club` objects the student has joined. Key logic includes `leaveClubInternal()`, a centralized helper method that handles safely removing a club from the fixed-size array and shifting elements to prevent null gaps.
- **`Employee` Class:** Extends `Person`. Manages baseline salary data and implements `calculateMonthlyAmount()` to return the sum of base salary and allowances as a negative value (representing a school expense).
- **`Teacher` & `AdminStaff` Classes:** Both extend `Employee`. `Teacher` introduces subject specialties and acts as a reference point for club supervision. `AdminStaff` introduces specific organizational titles. Both classes override `getRoleInfo()` to provide customized role descriptions.

3. Other Objects

- **`Classroom` Class:** A straightforward data model that aggregates room identification and capacity details.
- **`Club` Class:** Manages an activity group, maintaining an aggregation relationship with a `Teacher` (supervisor) and `Student` members in a

fixed-size array. It contains critical logic to add members, remove members by ID, and verify membership status before allowing structural changes.

4. The Core Management Systems

- **`School` Class:** The central operational class of the program. It maintains the arrays of people, clubs, and classrooms. Notable logical implementations include:
  - **Bidirectional-Synchronization:** Methods like `addStudentToClub()` and `removeStudentFromClub()` act as safe wrappers. They ensure that if a student joins a club, the system successfully updates *both* the `Club` object's roster and the `Student` object's personal list. If either update fails (e.g., due to array capacity), it reverts the action to maintain data integrity.
  - **EntityDeletion:** When `removePersonById()` or `removeClubByName()` is called, the class iterates through the necessary sub-arrays to safely dissolve relationships (e.g., removing a deleted student from their respective clubs, or removing a deleted club from all students' records).
  - **Recursion:** Implements `countStudentsRecursive()`, which traverses the `people` array without a loop to count how many `Student` objects currently exist in the system.
- **`SchoolTest` Class:** The main test program. It hardcodes initial sample data to populate the environment, then enters a loop of role-based switch-statement menus. It includes authentication logic that cross-references a user's input with the `School`'s database, dynamically showing different functionalities depending on whether the authenticated user is an admin, a teacher, a standard employee, or a student.

Conclusion

In Phase 1, we effectively established the core operational logic of the school management system. By utilizing inheritance trees, interfaces, and structured arrays, the program successfully fulfills the core principles of object-oriented design and creates a reliable foundation for future expansion.